

DocAuditor

Encoder-Only Document Intelligence for Legal, HR and Compliance Review

Student	Preeyas Tumulu
Subject	Generative AI
Assessment	Unit Test 01
Repository	github.com/PreeyasTumulu/DocAuditor_BERT

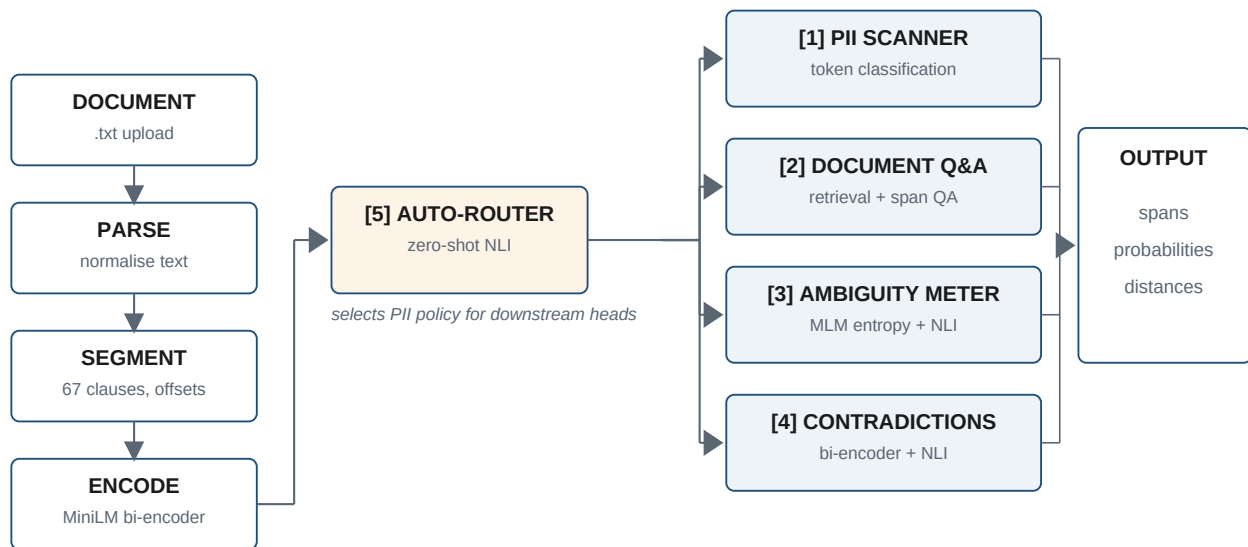
Thesis

Every feature in this system is a different output head on the same family of bidirectional encoders. No generative model is used at any point. The result is a **glass-box auditor**: every output is a character span, a calibrated probability, or a cosine distance - each traceable to the source text. An encoder has no decoder, so there is no mechanism by which it can invent a clause that is not there. That is the entire argument for this architecture in a regulated setting, where an unverifiable answer is worse than no answer.

Component	Deliverable
Pipeline	DocAuditor.ipynb - 36 cells; all 18 code cells executed with saved outputs, no errors
Interface	app.py - Streamlit; accepts TXT, PDF and DOCX
Corpus	3 documents, 54 annotated PII spans, 3 planted defects
Hardware	NVIDIA RTX 3050, 4 GB VRAM; CUDA 12.1, torch 2.5.1

1. System Architecture

A document is parsed, segmented into clauses, and encoded once. The router runs first and configures the remaining heads; the four analysis heads then operate independently over the shared clause set.



All five heads are output layers over BERT-family encoders. No decoder appears anywhere in the system.

1.1 Model stack

#	Feature	Model	Head
1	PII Scanner	iiiorg/piiranha-v1 (mDeBERTa-v3)	token classification
2	Document Q&A	all-MiniLM-L6-v2 + roberta-base-squad2	retrieval + span QA
3	Ambiguity Meter	bert-base-uncased + nli-deberta-v3-base	MLM entropy + NLI
4	Contradiction Detector	all-MiniLM-L6-v2 + nli-deberta-v3-base	bi-encoder filter + NLI
5	Document Auto-Router	nli-deberta-v3-base	zero-shot classification

1.2 Requirement mapping

Brief requirement	Feature	App tab	Input formats
Scan for personally identifiable information	PII Scanner	PII Scanner	TXT / PDF / DOCX
Ask plain-English questions of the document	Document Q&A	Ask the Document	TXT / PDF / DOCX
Writing assistant to test ambiguous language	Ambiguity Meter	Ambiguity Meter	free text or document
Feature of choice #1	Contradiction Detector	Contradictions	TXT / PDF / DOCX
Feature of choice #2	Document Auto-Router	Router	TXT / PDF / DOCX

DOCX tables are flattened row by row rather than read as paragraphs: contract schedules and HR forms keep much of their sensitive detail inside tables, which a paragraph-only reader drops silently. Scanned PDFs are rejected with an explanatory message - this pipeline reads text, not images, and OCR is out of scope.

Are these all BERT? Yes. RoBERTa is BERT with improved pretraining (no next-sentence prediction, dynamic masking). DeBERTa-v3 is BERT with disentangled attention and ELECTRA-style pretraining. MiniLM is a distilled BERT-family sentence encoder. All are bidirectional transformer encoders trained with a masked-language-model objective. Model identifiers are centralised in a single dictionary, so the entire stack can be swapped to vanilla bert - base - * checkpoints in one edit.

2. The Governing Constraint

BERT reads a maximum of 512 word-piece tokens. The test contract is roughly twelve times that limit. This single fact drives every significant design decision in the system.



Head	Consequence of the 512-token window
Document Q&A	Cannot read the document. Retrieves candidate clauses with a bi-encoder, then reads only the best few.
Contradiction	Cannot compare all pairs (2,211 for 67 clauses). Filters candidates by embedding similarity before the cross-encoder.
Auto-Router	Cannot see the whole document. Classifies from a representative excerpt sampling both the head and the middle.
PII Scanner	Slides an overlapping window across the text and maps every span back to absolute offsets; overlap prevents boundary truncation.

These are not workarounds bolted on afterwards. Retrieval, candidate filtering and windowing *are* the architecture.

3. Feature Design

3.1 PII Scanner (token classification)

Each word-piece receives a label from a 17-class PII tag set; adjacent tokens sharing a tag merge into a span. A deterministic regex layer runs alongside for structurally regular identifiers (e-mail, card numbers with a Luhn check, national IDs). The regex is a safety net, not a replacement: the model finds names and addresses no pattern can express, while the regex guarantees a card number is never lost to a tokenisation quirk. Recording which source found each span makes the model's real contribution measurable.

3.2 Document Q&A (retrieval + extractive span QA)

Two stages: a bi-encoder ranks clauses against the question, then a span-QA head decodes start and end logits over the best candidates. The answer is a pair of indices into the input, which is precisely why the model cannot fabricate: it can only point.

Known limitation - stated, not hidden

Extractive QA can only return text that literally exists in the document. Asked "what are the risks here?" it will fail, because that answer is written nowhere and would have to be composed. The mitigation is a confidence floor: below it the system answers "Not stated in document" rather than guessing. For a compliance tool an honest abstention is far more valuable than a plausible fabrication - which is exactly the failure mode a generative model would exhibit here.

3.3 Ambiguity Meter (MLM entropy + NLI)

Testing for ambiguous language sounds like it requires generation. It does not. Two encoder-native signals combine:

Masked-language-model entropy. Mask the term and measure the entropy of BERT's predictive distribution over the vocabulary. If context strongly determines the word, the distribution is sharp and entropy is low. A vague legal standard leaves the slot genuinely open, so entropy is high. This reads BERT's original pretraining objective directly, which is why it needs no fine-tuning.

$$H = - \sum \text{over vocabulary of } P(w \mid \text{context}) * \log P(w \mid \text{context})$$

NLI double-entailment. If a clause entails both a strict and a permissive reading, it is not merely vague but genuinely ambiguous - two parties could read it in good faith and reach opposite conclusions. High entropy alone means *vague*; high entropy with double-entailment means *ambiguous*, a stronger and more actionable finding.

3.4 Cross-Clause Contradiction Detector

Finds clause pairs that cannot both hold - the classic contract defect where one section says thirty days and another says fifteen. Comparing every clause with every other is quadratic: 2,211 pairs for 67 clauses, each requiring a full forward pass over a concatenated pair. The design mirrors retrieve-then-rerank:

Stage	Model	Purpose
1. Filter (cheap)	bi-encoder	Embed each clause once; keep only topically related pairs. A payment clause can only contradict another payment clause. Removes 97% of pairs.
2. Judge (costly)	NLI cross-encoder	Run on survivors only, in both directions, using the model's native contradiction output.

3.5 Document Auto-Router (zero-shot classification)

An NLI model scores how strongly the document entails each of several candidate descriptions - no training, no labelled data. The verdict is consequential rather than cosmetic: it selects the PII policy, so a national ID number is escalated to high severity in a medical record but not in a commercial contract. The excerpt samples the head *and* the middle of the document, because the opening of a contract and the opening of a medical record both look like a title block.

4. Evaluation

Because the corpus was built with synthetic PII injected at recorded offsets, the scanner can be *scored* rather than demonstrated. Offsets are exact by construction - captured during document assembly, not searched for afterwards - and the preparation script asserts every offset against the written file before exiting.

4.1 PII detection

Document	TP	FP	FN	Precision	Recall	F1
contract.txt	32	7	5	82.1%	86.5%	84.2%
medical_report.txt	9	7	3	56.3%	75.0%	64.3%

Document	TP	FP	FN	Precision	Recall	F1
hr_grievance_email.txt	13	4	4	76.5%	76.5%	76.5%
Overall	54	18	12	75.0%	81.8%	78.3%

Scoring is by span overlap rather than exact label match. The answer key uses FULLNAME where the model emits GIVENNAME and SURNAME separately; penalising that would measure a taxonomy difference, not a mistake.

4.2 Auto-router and contradiction detector

Measurement	Result
Router accuracy	3 / 3 documents correct (contract 95%, HR e-mail 99%, medical 65%)
Contradiction search space	67 clauses, 2,211 possible pairs
Pairs judged after filtering	61 (97.2% eliminated before the cross-encoder)
Findings at default setting	9, including the planted governing-law conflict

5. What the Evaluation Revealed

The corpus assumption was wrong, and the tool proved it

The first evaluation scored 57% precision on the contract. Inspecting every apparent false positive showed that **none of them were false**. They were real names, e-mail addresses, telephone numbers and street addresses belonging to identifiable people, already present in the CUAD source document.

The corpus had been built on the assumption that SEC filings are pre-redacted and therefore PII-free - which is why synthetic PII was injected in the first place. That assumption was false. The [* * *] markers redact commercially sensitive terms; the notice blocks, containing real contact details, were published untouched.

Each detection was manually verified against the source and added to the answer key. Precision rose from 57% to 75%, and F1 from 67% to 78% - not by tuning the model, but by correcting a flawed measurement.

A tool built to find personal data in documents found personal data that a regulatory filing process had missed, on real public data. That is the compliance use case, demonstrated by accident.

6. Calibrating the Contradiction Detector

The first working version reported **116 contradictions from 400 judged pairs**, nearly all scoring 1.00. No reviewer will read 116 findings, and spot-checking showed most were not contradictions at all.

This was not a bug. The label order, softmax and raw logits were all verified correct, and on clean sentence pairs the model is well calibrated (true contradiction 1.000, unrelated 0.000, entailment 0.000). The failure is domain mismatch: NLI is trained on short everyday sentences, while legal prose hedged with "notwithstanding" and party-specific scoping reads as contradictory to a model that has never seen a contract.

6.1 A hypothesis that was tested and rejected

Requiring contradiction in *both* directions appeared principled - entailment is directional, but mutual exclusivity should be symmetric. It reduced findings from 116 to 40 and scored every planted defect at 0.007-0.067. The rule would have destroyed every true detection while appearing to improve precision. These contradictions are strongly asymmetric in practice, and the idea was discarded on the evidence.

6.2 The effective lever, measured

Model failures concentrate in long clauses, so clause length is the useful control. The trade-off is exposed as a parameter rather than hidden behind a tuned constant:

Clause length cap	Threshold	Findings to review	Planted defects found
250 (default)	0.95	9	2 / 3
350	0.95	17	2 / 3
500	0.95	34	2 / 3
none	0.95	102	3 / 3
none	0.99	68	2 / 3

The third defect is genuinely missed at the default setting: its conflicting source language sits in a 1,145-character clause, above the cap that makes the output reviewable. Which operating point is correct depends on whether a human reviews every hit or triages a shortlist.

7. Limitations

Limitation	Consequence
Contradiction precision is weak	Roughly one real defect per nine findings. A triage aid, not an oracle. Domain-tuned NLI would be the fix.
One planted defect missed	Recall failure at the default clause cap; recoverable only by accepting ~102 findings.
Extractive QA cannot compose	No summarisation, no synthesis, no "what are the risks". Abstains rather than guessing.
Zero-shot NLI is not domain-tuned	Thresholds are heuristic rather than calibrated against a labelled legal corpus.
Similarity floor bounds recall	A contradictory pair below the floor is never judged. Lowering it costs quadratically more compute.
Medical precision is lowest	56.3% - clinical prose contains dates and identifiers that resemble PII structurally.

8. Data Provenance and Ethics

Document	Source	Licence
contract.txt	CUAD - real SEC EDGAR commercial services agreement	CC BY 4.0
medical_report.txt	MTSamples - real de-identified clinical transcription	CC0
hr_grievance_email.txt	Synthetic, authored for this project	-

Why real sources. Real documents provide authentic language; synthetic PII provides a ground-truth answer key. Using only generated documents would yield an answer key but unrealistically clean prose, giving the ambiguity meter nothing genuine to analyse. Using only real documents would give realistic language but no way to measure anything.

Why the HR thread is synthetic. Real grievance correspondence is not public, for good reason, and the widely used real e-mail corpora contain personal data whose subjects never consented to its release. Authoring that document was the correct choice rather than a convenient one.

Disclosure. As documented in section 5, `contract.txt` contains real personal data inherited from the public SEC filing. It is retained because removing it would destroy the most informative result in the evaluation, and because the data is already published in a public regulatory database.

9. Future Work

Domain-tuned NLI. Fine-tuning the entailment head on contract-specific clause pairs would replace heuristic thresholds with calibrated ones and directly address the contradiction detector's precision, the system's weakest point.

Playbook deviation detection. Embedding each clause against a library of company-approved clauses would extend the system from finding defects to proposing the approved alternative - the capability that separates a review tool from a drafting assistant.

Hierarchical segmentation. Splitting long clauses into sentence-level units for contradiction analysis would recover the missed defect without accepting the false-positive rate of an uncapped search.

References: Devlin et al., *BERT* (2019). He et al., *DeBERTa* (2021). Hendrycks et al., *CUAD* (2021). Reimers & Gurevych, *Sentence-BERT* (2019). Williams et al., *MultiNLI* (2018).